

Huntio

Find it. Show it. Win it.

Phase 1 Project Plan

What we are building, pages, tech stack, APIs, system design and file structure.

Section	Contents
1	What we are building
2	Pages
3	Tech stack
4	APIs (REST and Socket events)
5	High-level system design
6	Low-level system design
7	Phase 1 file structure
8	Build order

1. What we are building

Huntio is a Skribbl-style party game, but instead of drawing, players hunt for real objects. Friends join a room with a Room ID and turn on their camera. The game shows a message such as **Find a bottle**. Everyone searches their own room, shows the object to their camera, and an AI model (YOLO) detects it. Points go to the fastest players, and a room leaderboard is updated after every round.

Game flow

- Home screen: enter a name, then create a room or join with a Room ID. No login.
- Lobby: everyone turns on their camera and clicks **Ready**.
- When everyone is ready, a 10-second countdown starts, then the game begins.
- Each round lasts 60 seconds. The item to find is written at the top of the screen.
- The AI detects the object. Only the AI judges, there is no player voting.
- Points by finish order: 1st 100, 2nd 80, 3rd 60, 4th 40, others 20, plus a small speed bonus.
- After each round the leaderboard on the left of the screen updates. There are 5 rounds.
- After round 5 the final leaderboard is shown, with **Leave room** and **Play again**.

Phase 1 decisions

Topic	Decision
Round timer	60 seconds. Round ends early if every player has found the item.
Scoring	100 / 80 / 60 / 40 / 20 by rank, plus a speed bonus of 0 to 20.
Detection	YOLO in the browser, using its 80 built-in objects.
Prompt	Text at the top of the screen. No voice.
Judge	AI only. No player voting.
Players	2 to 8 per room.
Camera	Used only for detection. Players do not see each other's video.
Rooms	Room ID is generated automatically. Room locks when the game starts.
Host	Starts the game flow. If the host leaves, the next player becomes host.
Disconnects	Player is removed, their score stays on the leaderboard.
Out of scope	Login, database, global ranking, power-ups, game modes, shared video.

Phase 1 item pool (from YOLO's 80 classes)

Level	Items
Easy	bottle, cup, book, remote, keyboard, mouse, clock, backpack, spoon, fork, bowl
Medium	toothbrush, umbrella, vase, potted plant, teddy bear, handbag, tie, scissors, sports ball, laptop
Left out	cell phone (cannot be shown to its own camera), fruit (not always at home), knife (safety), outdoor objects

2. Pages

Phase 1 has **4 pages**. The game page changes what it shows by state, so there are no extra pages for countdown or round results.

Page	Route	Purpose
Home	/	Enter a name. Create a room or join with a Room ID.
Lobby	/lobby/[roomId]	Shows Room ID, player list, camera preview and Ready button. The 10-second countdown runs here.
Game	/game/[roomId]	Item at the top, 60-second timer, player's camera with AI detection, leaderboard on the left.
Results	/results/[roomId]	Final leaderboard and podium. Leave room or Play again.

Game page layout

+-----+ Round 2/5 FIND A BOTTLE 0:42 +-----+		
Leaderboard		
1. Rahul 180	Your camera	
2. Priya 140	(AI detecting...)	
3. You 100		
4. Amit 60	[status: found it!]	
+-----+		

States inside the Game page

- **Round active:** item, timer, camera and detection.
- **Round results (about 5 seconds):** popup showing who found it and points gained; the left leaderboard updates.
- After round 5, everyone is moved to the Results page.

3. Tech stack

Two decisions keep phase 1 simple. First, YOLO runs **inside the player's browser**, so there is no separate YOLO or FastAPI backend: it is free, fast, and camera images never leave the device. Second, rooms are held in an **in-memory Map**, so Redis is not needed yet.

Frontend

Technology	What it does	Why we use it
Next.js (App Router)	Frontend framework and page routing	Already set up; /lobby/[roomId] routes are built in
TypeScript	Typed JavaScript	Catches mistakes, especially in socket event data
Tailwind CSS	Styling	Fast layout building
socket.io-client	Realtime connection	Live player list, round start, scores
Zustand	Small state manager	Shares room and game state across pages
getUserMedia	Browser camera API	Built in, nothing to install
onnxruntime-web	Runs AI models in the browser	Runs the YOLO model locally
YOLO11n / YOLOv8n (.onnx)	Object detection model	Small (about 6-12 MB), fast, 80 common objects

Backend

Technology	What it does	Why we use it
Node.js + Express	Web server	Simple; one language across the project
Socket.IO	Realtime rooms and events	Rooms are built in, so joining by Room ID is easy
In-memory Map	Stores rooms, players, scores	Fast and simple for a single server
dotenv, cors	Config and cross-origin access	Frontend (port 3000) talks to server (port 4000)
tsx	Runs TypeScript in development	Auto-restarts on save

Tools and deployment

Technology	What it does	Why we use it
Python + ultralytics (one time)	Exports YOLO to ONNX	Run once on your computer to produce the model file
ngrok	HTTPS link to local app	Phone browsers only allow the camera on HTTPS
Vercel + Render/Railway	Hosting	Free or cheap tiers; frontend on Vercel, server on Render/Railway

Not used in phase 1

- **FastAPI / separate YOLO server:** would send every frame to the server, adding cost and delay. Useful later for items YOLO does not know (CLIP) or server-side checks.
- **Redis:** needed only when running several servers or surviving restarts.
- **LiveKit / WebRTC:** only needed if players should see each other's video.
- **Database:** no accounts and no history to store.

4. APIs

Phase 1 uses **2 REST endpoints**, **6 client-to-server socket events** and **11 server-to-client socket events**.

REST

Method	Endpoint	Purpose
GET	/health	Checks that the server is running.
GET	/api/rooms/:roomId	Tells whether a room exists, is full or has started. Used by the Home page before joining.

Socket events: client to server

Event	Data	What it does
room:create	{ nickname }	Creates a room with a generated 6-character ID; sender becomes host.
room:join	{ roomId, nickname }	Adds the player. Rejected if the room is missing, full or started.
room:leave	none	Removes the player. Host moves to the next player if needed.
player:ready	{ ready }	Marks the player ready or not ready.
round:found	{ label, confidence }	Browser reports a detected item. Server validates and awards points.
game:playAgain	none	Host only. Resets scores and returns everyone to the lobby.

Socket events: server to client

Event	Data	When it fires
room:updated	{ roomId, players[], status }	Someone joins, leaves or changes ready state.
countdown:start	{ startsAt, seconds }	Everyone is ready; 10-second countdown begins.
countdown:cancel	none	Someone un-readies or leaves during the countdown.
game:started	{ totalRounds }	Moves everyone to the Game page.
round:start	{ round, totalRounds, item, endsAt }	New round begins with the item to find.
round:playerFound	{ playerId, nickname, points, rank }	Shows others that a player found the item.
round:end	{ round, results[], scores[] }	Timer ended or everyone found it.
leaderboard:update	{ scores[] }	After each round, updates the left panel.
game:over	{ finalLeaderboard[] }	After round 5; moves everyone to Results.
room:reset	{ players[] }	After Play again; back to the lobby.
error	{ message }	Any rejected action.

Event flow

```
Lobby:  room:create / room:join  -> room:updated
        player:ready (all ready) -> countdown:start (10s) -> game:started

Game:   round:start -> players find item -> round:found
        round:playerFound (live, to others)
        round:end + leaderboard:update          (repeat x5)

End:    game:over -> Results page
        game:playAgain -> room:reset      or      room:leave
```

Rules the server enforces

- The server owns the timer. It sends **endsAt**; the browser only displays the countdown.
- One win per player per round. A repeat **round:found** is ignored.
- Submissions after **endsAt** get no points.
- A wrong label gets no points; it must match the current item.
- Only the host can use **game:playAgain**.

5. High-level system design



Main components

Component	Responsibility
Next.js client	Shows screens, reacts to server events, runs the camera and the AI.
Detector (browser)	Runs YOLO on camera frames and decides when the target item is clearly visible.
Socket.IO server	Single source of truth for rooms, timers, scores and rankings.
In-memory store	Holds all room state. Lost on restart, which is acceptable in phase 1.

Core principle

The server is the authority; the browser is only a reporter. The browser says "I saw a bottle". The server decides whether it counts: right item, round still open, first time this player found it, and what rank and points they get.

One round, end to end

```
Server: round:start {item:"bottle", endsAt: T+60s} -> all clients
Client: camera -> YOLO -> bottle seen in 3 frames in a row
Client: round:found {label:"bottle"} -> server
Server: validate -> rank = foundCount+1 -> points -> store
Server: round:playerFound -> all clients (live toast)
Server: (all found OR timer ends) round:end + leaderboard:update -> all clients
        wait 5s, then next round:start (or game:over after round 5)
```

Deployment

Vercel (Next.js) --HTTPS--> Browser --WSS--> Render/Railway (Express + Socket.IO)

Camera access needs HTTPS, so both frontend and server must be served over HTTPS. For phone testing on a local network, use ngrok.

6. Low-level system design

6.1 Server state machine

```
LOBBY --all ready--> COUNTDOWN (10s) --> ROUND_ACTIVE (60s)
  ^                               | someone un-readies/leaves |
  |                               |--> back to LOBBY              v
  |                               ROUND_RESULTS (5s)
  |----- playAgain ----- FINISHED <-- round = 5 -----' | round < 5: next round
```

State	Allowed actions
LOBBY	join, leave, ready
COUNTDOWN	leave, un-ready (cancels the countdown)
ROUND_ACTIVE	round:found, leave
ROUND_RESULTS	leave
FINISHED	playAgain (host only), leave

Joining is allowed only in LOBBY, so the room locks once the countdown starts.

6.2 Data structures (server)

```
type RoomStatus = "lobby" | "countdown" | "round" | "results" | "finished";

interface Player {
  id: string;           // socket id
  nickname: string;
  isHost: boolean;
  ready: boolean;
  score: number;
}

interface FoundEntry { rank: number; timeMs: number; points: number }

interface RoundState {
  number: number;           // 1..5
  item: Item;
  startedAt: number;        // ms epoch
  endsAt: number;
  found: Map<string, FoundEntry>; // playerId -> entry (one per player)
}

interface Room {
  id: string;
  status: RoomStatus;
  players: Player[];        // max 8
  currentRound: RoundState | null;
  usedItems: string[];      // no repeats in one game
  timers: { countdown?: Timeout; round?: Timeout; next?: Timeout };
}

const rooms = new Map<string, Room>();
```

6.3 Key algorithms

Room ID. 6 characters from ABCDEFGHJKLMNPQRSTUVWXYZ23456789 (no 0, O, 1, I); regenerated if it already exists.

Starting the countdown. After every player:ready, check that there are at least 2 players and all are ready. If so, set status to countdown and start a 10-second timeout. Any leave or un-ready clears the timeout and emits countdown:cancel.

Handling round:found.

```
1. room.status == "round" ?           else ignore
2. now <= round.endsAt ?               else ignore
3. label == round.item.detectorLabel ? else ignore
4. round.found.has(player.id) ?       then ignore (already scored)
```

```

5. rank    = round.found.size + 1
6. points = scoring(rank, endsAt - now)
7. player.score += points; store the entry
8. emit round:playerFound
9. if found.size == players.length -> endRound() early

```

Scoring.

```

const RANK_POINTS = [100, 80, 60, 40, 20];

function scoring(rank: number, msLeft: number) {
  const base = RANK_POINTS[Math.min(rank, 5) - 1];
  const speedBonus = Math.floor((msLeft / 60000) * 20); // 0..20
  return base + speedBonus;
}

```

Ending a round. Clear the round timer, set status to results, emit round:end with each player's points gained (0 if not found) and leaderboard:update sorted by score. After 5 seconds, start the next round, or emit game:over if it was round 5.

Leaving. Remove the player; reassign host to the first remaining player if needed; delete the room when empty. If mid-round and every remaining player has already found the item, end the round. If fewer than 2 players remain mid-game, end the game and show the leaderboard.

Ties. Sort by score, then by total solve time (lower is better).

6.4 Client structure

Zustand store: me, roomId, players, status, round, totalRounds, item, endsAt, scores, foundBy, countdownEndsAt, finalLeaderboard.

One hook, **useGameSocket**, listens to every server event and writes to the store; pages only read from it, so all state changes happen in one place.

On-screen timers. The client computes endsAt minus the current time every 250 ms. Because server and client clocks can differ, the client asks for the server time once on connect and stores the offset.

6.5 Detection pipeline (browser)

```

Camera video -> draw to hidden canvas (640x640, letterboxed)
-> tensor [1,3,640,640] float32, normalized 0..1
-> onnxruntime-web.run() (WebGPU / WASM)
-> output [1,84,8400]
-> decode boxes + class scores
-> NMS (non-max suppression, written by us)
-> filter: label == target AND confidence >= 0.55
-> confirm: seen in 3 consecutive frames
-> emit "round:found"

```

- Run about every 300-400 ms, not every frame, to save phone battery.
- Runs only during ROUND_ACTIVE, and stops once the player has found the item.
- YOLOv8/11 ONNX output has no built-in NMS, so we write the decode and NMS step ourselves.
- Use the nano model (about 6-12 MB), loaded once in the lobby so rounds start instantly.
- Use a Web Worker if the UI lags during inference.
- item.detectorLabel must match COCO names exactly (for example: potted plant, teddy bear).

6.6 Edge cases

Case	Handling
Room not found, full or started	room:join returns an error; Home page shows it.
Player refreshes mid-game	Socket ID changes, so they are treated as left in phase 1. Reconnect token later.
Host leaves	Next player becomes host.
Fewer than 2 players mid-game	Game ends and shows the leaderboard.

Case	Handling
Duplicate round:found	Ignored (one entry per player per round).
Late round:found	Ignored after endsAt.
Camera permission denied	Ready is disabled and a message explains why.
Model fails to load	Lobby shows an error and blocks Ready.
Nobody finds the item	Round ends at 60 seconds; everyone gets 0.

6.7 Security and fairness (phase 1 level)

- The server owns time, rank and score, and ignores any score sent by the client.
- Validate input: nickname length 1-16, Room ID format, and that each event is allowed in the current state.
- Rate-limit socket events per connection; restrict CORS to the frontend URL.
- Known weak spot: a determined player could fake round:found or hold up a photo. Acceptable for a friends game; server-side checks come in a later phase.

6.8 Scaling and testing

- One Node process handles thousands of sockets, far more than needed for friends playing.
- To scale later: Socket.IO Redis adapter with sticky sessions, and room state moved into Redis. RoomManager and GameEngine keep storage swappable.
- Test plan: unit tests for scoring, itemPicker and state transitions; two-browser test on localhost; detection test with the item list held to the camera (tune the 0.55 threshold); phone test through ngrok.

7. Phase 1 file structure

Two separate projects side by side. The server sits **outside** the Next.js project so Next.js does not try to compile server files. Files marked (done) already exist from your setup.

Frontend: scavengerhunt (Next.js, no src folder)

```
scavengerhunt/
|-- app/
|   |-- layout.tsx                (done)
|   |-- globals.css              (done)
|   |-- page.tsx                 Home: create / join room
|   |-- lobby/[roomId]/page.tsx  Lobby: players, camera, Ready, countdown
|   |-- game/[roomId]/page.tsx   Game: item, timer, camera, leaderboard
|   |-- results/[roomId]/page.tsx Final leaderboard, Leave / Play again
|
|-- components/
|   |-- lobby/
|   |   |-- PlayerList.tsx
|   |   |-- ReadyButton.tsx
|   |   |-- Countdown.tsx
|   |-- game/
|   |   |-- ItemBanner.tsx        "Find a bottle"
|   |   |-- Timer.tsx
|   |   |-- CameraView.tsx       video + detection status
|   |   |-- FoundToast.tsx       "Rahul found it!"
|   |   |-- RoundResultModal.tsx
|   |-- leaderboard/
|   |   |-- Leaderboard.tsx      left panel during the game
|   |   |-- Podium.tsx           results page
|
|-- hooks/
|   |-- useCamera.ts             getUserMedia stream
|   |-- useDetector.ts           runs YOLO loop, reports matches
|   |-- useGameSocket.ts         subscribes to all server events -> store
|
|-- services/
|   |-- socket.ts                socket.io client singleton
|   |-- api.ts                   GET /api/rooms/:roomId
|   |-- detector/
|   |   |-- yolo.ts              load model, run inference
|   |   |-- postprocess.ts       decode boxes + NMS
|   |   |-- labels.ts            COCO class names
|
|-- store/
|   |-- gameState.ts             Zustand store
|
|-- types/
|   |-- game.ts                  Player, Room, Round, event payloads
|
|-- utils/
|   |-- constants.ts             timers, thresholds, round count
|   |-- time.ts                  server clock offset helper
|
|-- public/
|   |-- models/yolo11n.onnx      exported model file
|
|-- .env.local                    NEXT_PUBLIC_SERVER_URL=http://localhost:4000
|-- package.json                  (done)
|-- next.config.ts                (done)
|-- tsconfig.json                 (done)
```

Backend: huntio-server (Node + Express + Socket.IO)

```
huntio-server/
|-- src/
|   |-- index.ts                 Express + Socket.IO bootstrap
|   |-- types.ts                 Player, Room, RoundState, events
|   |
|   |-- socket/
|   |   |-- roomHandlers.ts      create, join, leave, ready
|   |   |-- gameHandlers.ts     round:found, playAgain
```

```

| |
| |-- game/
| |   |-- RoomManager.ts      create/get/delete rooms, ID generation
| |   |-- GameEngine.ts       state transitions + timers
| |   |-- scoring.ts           points formula
| |   |-- itemPicker.ts        random item, no repeats
| |
| |-- routes/
| |   |-- rooms.ts             GET /api/rooms/:roomId, GET /health
| |
| |-- data/
| |   |-- items.ts             phase 1 item pool
| |
|-- .env                       PORT=4000, CLIENT_URL=http://localhost:3000
|-- package.json
|-- tsconfig.json

```

One-time model export (run on your computer)

```

pip install ultralytics
yolo export model=yolo11n.pt format=onnx imgsz=640
# copy yolo11n.onnx into scavengerhunt/public/models/

```

8. Build order

The fake-button step lets you test rooms, timers and scoring before the AI part exists.

Step	Task	Result
1	Server: RoomManager, GameEngine, socket handlers	Rooms, ready, countdown, rounds and scoring work
2	Home page and Lobby page with countdown	Two browsers can create, join and ready up
3	Game page with a fake "I found it" button	Full 5-round flow works without AI
4	Replace the button with real YOLO detection	Camera detects the item and reports it
5	Results page and Play again	Complete game loop
6	Test on phones via ngrok, tune the detection threshold	Ready for friends to play

Later phases (not in phase 1)

- Shared video, game modes, power-ups, global leaderboard and accounts.
- FastAPI service with CLIP for items YOLO does not know, and server-side checks against cheating.
- Redis for scaling to several servers and reconnecting after a refresh.